The Journal of Open Source Software

RealPaver 1.1: A C++ Library for Constraint Programming over Numeric or Mixed Discrete-Continuous Domains

Raphaël Chenouard [1*¶] **and Laurent Granvilliers** [1*]

1 Nantes Université, École Centrale Nantes, CNRS, LS2N, UMR 6004, Nantes, France ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 20 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Constraint Programming (CP) is a paradigm for solving constraint satisfaction and optimization problems (Rossi et al., 2006). Although CP mainly addresses combinatorial problems, it can also handle continuous problems by approximating real numbers with intervals (Benhamou & Granvilliers, 2006).

RealPaver is a C++ library for CP over numeric or mixed discrete-continuous domains. Constraint Satisfaction Prblems (CSPs) can be described either in C++ with the API, or in a text file using the syntax of RealPaver specific language. Then, they can be solved using the C++ API or using the CSP solver from the command-line. The CSP solver is pre-configured, but various parameters can be modified in another text file.

With respect to the first version of the software developed twenty years ago (Granvilliers & Benhamou, 2006), this new library incorporates new types of variables and constraints, new algorithms, a clean object-oriented architecture, the management of parameters, Meson Build as build engine (Pakkanen, 2025), an interface with third-party softwares and a C++ API. It achieves performances equivalent to the competing library Ibex ("Ibex-Lib," 2025) for pure continuous problems.

Statement of need

CP associates a rich modeling language with powerful solving techniques. The main algorithm behind the RealPaver solver is a branch-and-prune (B&P) that implements a complete search to find all the solutions of a given problem (Chabert & Jaulin, 2009; Van Hentenryck et al., 1997). The branching component separates a problem into sub-problems easier to solve. The pruning or contracting component aims at reducing the region delimited by a sub-problem. RealPaver relies on the GAOL interval library (Goualard, 2020) to ensure rigorous computations.

This technology has been applied with success in many fields of engineering like automatic control (Jaulin et al., 2001), preliminary design (Yvars & Zimmer, 2021) and robotics (Merlet, 2004).

This library can be used by anyone wanting to compute sets of solutions for numerical or mixed discrete-continuous constraint satisfaction problems. It can also be used to prove infeasibility or existence of solutions thanks to the B&P algorithm and interval analysis (Moore et al., 2009). Since the library contains most of the state-of-the art algorithms relating to CP over intervals, it can also be used by researchers in this field to define new algorithms.

Overview

The RealPaver libray provides a CSP solver relying on a modern C++ API. Problems can be defined through the object-oriented API or using a domain specific language.

Modeling language

There are three types of variables:

- A boolean variable has domain $\{0, 1\}$;
- An integer variable can take values from a set of integers;
- A real variable lies in a union of intervals.

The language defines several types of constraints:

- An arithmetic constraint involves the usual operations over the reals and relations from the set $\{=, \leq, \geq\}$;
- $G \rightarrow B$ is a conditional constraint where a constraint B (body) is activated when a constraint G (guard) holds true;
- $table(X, S)$ is a table constraint where X is a vector of variables and S is a set of valid assignments for X;
- $piecewise(x, \{I_k \rightarrow C_k\}_k)$ is a piecewise constraint where a constraint C_k is activated when the variable x lies in the interval I_k.

Search strategies

The B&P algorithm creates a search tree by recursively dividing the initial region, i.e. the Cartesian product of variable domains. The search strategy is responsible for the selection of the next node to explore and the selection of a variable in this node. The domain of this variable is then split in several parts, hence generating sub-nodes.

- Node selection. Several strategies are implemented., e.g. depth-first search, breadth-first search and distant-most-depth-first search (Chenouard et al., 2009).
- Variable selection. It is possible to select e.g. discrete variables first as done in integer linear programming, discrete variables with the smallest domains, continuous variables with the largest domains or the greatest impacts on the constraints (Trombettoni et al., 2011).

Contraction algorithms

The pruning component of the B&P algorithm combines several techniques.

- Contraint propagation based on HC4 or BC4 operators (Benhamou et al., 1999) and the ACID algorithm (Neveu et al., 2015);
- The interval Newton operator for nonlinear systems of equations (Moore et al., 2009);
- Linear methods applied to affine or Taylor relaxations of nonlinear problems (Ninin et al., 2015; Trombettoni et al., 2011);
- Specific algorithms for the non-arithmetic or global constraints.

Parameters and RealPaver customization

RealPaver integrates classes to handle three types of parameters: double-valued, integer-valued or string-valued parameters.

All existing parameters, with their default value, are defined in the class `Params`. This class organizes them using 10 categories: `General`, `Branch`, `Split`, `Contract`, `Polytope`, `Acid`, `LocaOptim`, `LinearSolver`, `Interval` and `NoCat`.

DRAFT

Each parameter is defined by a name, a category, a description as a string, a domain (interval for doubles, range for integers and enumerated list of strings), and a value. All these elements are summarized in the default parameter file that is generated with the B&P solver executable. For each parameter in this file, a comment before the parameter gives its description and possible values. Below is an extract of the general category:

```
# Time limit in seconds
# Range of possible real values: [0, inf]
TIME_LIMIT = 3600

# Display of regions
# List of possible values:
# - STD: standard display with one variable and its domain per line
# - VEC: display as a vector of domains
DISPLAY_REGION = STD

# Number of digits used to print floating-point numbers and intervals
# Range of possible int values: [1..16]
FLOAT_PRECISION = 12

# Absolute tolerance on the width of an interval domain of a constrained
# variable
# Range of possible real values: [0, inf]
VAR_ABS_TOL = 1e-08
```

The B&P behavior can be controlled by changing various parameters, like:

- `SEARCH_STATUS` (`LIMITED` or `COMPLETE`): in the case of limited search, additional parameters act to stop the solving process (`TIME_LIMIT`, `SOLUTION_LIMIT`, `NODE_LIMIT`, `DEPTH_LIMIT`), otherwise they are ignored to compute the whole set of solutions that reaches the tolerances on all the variables.
- `BP_NODE_SELECTION` (`DFS`, `BFS`, `DMDFS`): defines the global exploration strategy

Additionally, more specific behaviors can be configured:

- branching strategies (`SPLIT_SELECTION`, `SPLIT_INTERVAL_POINT`);
- propagation and contraction algorithms (`PROPAGATION_BASE` selects `HC4` or `BC4`, `PROPAGATION_WITH_NEWTON`, etc.);
- parameters dedicated to the selected mechanisms, like the tolerance of the interval Newton contraction algorithm (`NEWTON_TOL`).

Moreover, the section about the parameters in the documentation (processed by MkDocs) is generated from the default parameter file, using the python script `doc/gen_params_doc.py`.

Building system and requirements

The meson build system is used to orchestrate the configuration, the building of the library and the generation of `rp_solver` (the CSP solver executable). The user can select one of the supported linear solving libraries (Coin-or CLP, HiGHS, SoPlex and Gurobi) and can activate assertions, logging or the generation of the documentation, directly as meson command line options.

The current building system does not install dependencies or third party softwares. The user has to install, by its own, the GAOL interval library (Goualard, 2020) and one of the supported linear solving library, as well as MkDocs if the building of the documentation is activated. Meson build system supports unit tests and eases their definition and execution. Numerous executable tests cover internal mechanisms at various levels from interval arithmetic to piecewise constraints and polytope contractors.

DRAFT

Running Example

The following problem is a simplified sizing problem of an electric coil. Somes variables are continuous (with interval domains), and the others are discrete (an interval of integers or enumerated sets of values). The material of wires is chosen from a table describing its characteristics (density and electical conductivity):

```
Constants
I = 0.5,                        # current in A
mu_0 = 4*PI*1e-7;

Variables
D in [0.01,0.1],                # average diameter of coil in m
l in [0.01,0.1],                # length of coil in m
d in [0.005,0.01],              # diameter of wire in m
N integer in [10,1000],         # number of spirals
L in [0.01,0.1],                # self inductance of coil in H
rho in {2700,8800},             # material density in kg/m3
gamma in {37.7e6, 59.6e6};      # material conductivity in S/m

Aliases
  r_int = 4*N*D/(gamma*d^2),    # coil resistance in Ohm
  phi_w = mu_0*N*PI*D^2*I/(4*l),
  P_j = r_int*I^2,              # Joule losses in W
  M = rho*PI*D*l*d;             # coil mass in kg

Constraints
  L == N*phi_w,
  P_j <= 5,
  M <= 0.1,
  table({rho,gamma},{
    2700, 37.7e6,
    8800, 59.6e6
  });
```

Solving this problem can be done by writting the problem in a text file (for instance `coil.rp`), then running `rp_solver` on this file from the commande line:

```
rp_solver coil.rp -p params.txt
```

The `-p` is optional and allows to customize the parameters using a text file (here `params.txt`).

Using a time limit of 1 second and an absolute precision for variables of 0.001, the output on the command line is:

```
################################################################################
Realpaver 1.1.0 CSP solver
################################################################################
Files
   Input file......................... ./coil.rp
   Output file........................ coil.sol
################################################################################
Preprocessing
   Time............................... 0.000 (seconds)
   Status............................. checked
   Number of variables fixed........... 0
   Number of inactive constraints...... 1
################################################################################
```

DRAFT

```
Solving
   Time.............................. 1.145 (seconds)
   Number of nodes.................... 3341
   Search status...................... partial
   Solution status.................... no proof certificate
   Number of solutions................ 1662
   Time limit enabled................. 1.000 (seconds)
   Number of pending nodes............ 9
################################################################################
```

Since this problem is under-constrained, a huge number of boxes may be computed to pave the feasible space. In this example, the time limit stopped `rp_solver`, thus leading to a partial exploration with 3,341 computed solutions and 9 pending boxes (not fully processed boxes).

The set of computed solutions is stored in the text file coil.sol. It also contains the hull of pending boxes and the summary of the solving process and the used values for parameters.

Acknowledgements

References

Benhamou, F., Goualard, F., Granvilliers, L., & Puget, J.-F. (1999). Revising Hull and Box Consistency. *ICLP 1999, Proceedings of the 16th International Conference on Logic Programming, Cambridge, MA, USA*, 230–244. https://doi.org/10.7551/mitpress/4304.003.0024

Benhamou, F., & Granvilliers, L. (2006). *Continuous and Interval Constraints* (F. Rossi, P. van Beek, & T. Walsh, Eds.; Vol. 2, pp. 571–603). Elsevier. https://doi.org/10.1016/S1574-6526(06)80020-9

Chabert, G., & Jaulin, L. (2009). Contractor Programming. *Artificial Intelligence*, *173*, 1079–1100. https://doi.org/10.1016/j.artint.2009.03.002

Chenouard, R., Goldsztejn, A., & Jermann, C. (2009). Search Strategies for an Anytime Usage of the Branch and Prune Algorithm. In C. Boutilier (Ed.), *IJCAI 2009, proceedings of the 21st international joint conference on artificial intelligence, pasadena, california, USA, 2009* (pp. 468–473). http://ijcai.org/Proceedings/09/Papers/085.pdf

Goualard, F. (2020). GAOL (Not Just Another Interval Library). In *GitHub repository*. https://github.com/goualard-f/GAOL; GitHub.

Granvilliers, L., & Benhamou, F. (2006). Algorithm 852: RealPaver: An Interval Solver using Constraint Satisfaction Techniques. *ACM Trans. Math. Softw.*, *32*(1), 138–156. https://doi.org/10.1145/1132973.1132980

Ibex-lib. (2025). In *GitHub repository*. https://github.com/ibex-team/ibex-lib; GitHub.

Jaulin, L., Kieffer, M., Didrit, O., & Walter, É. (2001). *Applied Interval Analysis with Examples in Parameter and State Estimation, Robust Control and Robotics*. Springer. https://doi.org/10.1007/978-1-4471-0249-6

Merlet, J.-P. (2004). Solving the Forward Kinematics of a Gough-Type Parallel Manipulator with Interval Analysis. *The International Journal of Robotics Research*, *23*(3), 221–235. https://doi.org/10.1177/0278364904039806

Moore, R. E., Kearfott, R. B., & Cloud, M. J. (2009). *Introduction to Interval Analysis*. SIAM. https://doi.org/10.1137/1.9780898717716

Neveu, B., Trombettoni, G., & Araya, I. (2015). Adaptive Constructive Interval Disjunction: Algorithms and Experiments. *Constraints*, *20*(4), 452–467. https://doi.org/10.1007/

DRAFT

s10601-015-9180-3

Ninin, J., Messine, F., & Hansen, P. (2015). A Reliable Affine Relaxation Method for Global Optimization. *OR: A Quarterly Journal of Operations Research*, *13*(3), 247–277. https://doi.org/10.1007/s10288-014-0269-0

Pakkanen, J. (2025). The Meson Build system. In *GitHub repository*. https://github.com/mesonbuild/meson; GitHub.

Rossi, F., van Beek, P., & Walsh, T. (Eds.). (2006). *Handbook of Constraint Programming* (Vol. 2). Elsevier. https://doi.org/10.1016/S1574-6526(12)70003-2

Trombettoni, G., Araya, I., Neveu, B., & Chabert, G. (2011). Inner Regions and Interval Linearizations for Global Optimization. *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, 99–104. https://doi.org/10.1609/aaai.v25i1.7817

Van Hentenryck, P., McAllester, D., & Kapur, D. (1997). Solving Polynomial Systems Using a Branch and Prune Approach. *SIAM Journal on Numerical Analysis*, *34*(2), 797–827. https://doi.org/10.1137/S0036142995281504

Yvars, P.-A., & Zimmer, L. (2021). Integration of Constraint Programming and Model-Based Approach for System Synthesis. *IEEE International Systems Conference, SysCon 2021, Vancouver, BC, Canada, April 15 - May 15, 2021*, 1–8. https://doi.org/10.1109/SYSCON48628.2021.9447096

DRAFT